

Gooder Golf Writeup

Gooder Golf is a golf game improvement plan. I am completely addicted to golf and love to find ways to improve my game. When taking a lesson recently, the first thing the instructor asked was, "What's the biggest part of your game that's holding you back?" I don't always know the answer to this question, and without the golf instructor, I'm even further lost in how to best improve my game. This app is targeted towards mid to high handicap players who are looking to improve their game, providing them with a practice plan to lower their scores.

System Architecture

The system uses a two-tier Claude architecture via Anthropic's API. An orchestrator agent (Claude Sonnet 4.6) coordinates five specialist agents (Claude Haiku 4.5) through tool-calling: stat interpreter, mental game analyzer, course context lookup, practice plan generator, and reviewer. Each specialist is scoped to a single task and uses Haiku for cost efficiency.

The user input flows from a React frontend to a TanStack Start server route that invokes the orchestrator pipeline. The orchestrator dispatches tool calls to each specialist, which call Anthropic's API. A course lookup tool integrates with a golf course RapidAPI for real course data; if it fails, the system falls back to generic guidance. The reviewer evaluates the draft against a quality rubric and can nudge one revision before returning the final markdown plan.

Agentic Patterns

As stated above, the system has an orchestrator agent using a model, not code to call tools. The orchestrator has a general pattern for what agents to call, but ultimately makes decisions about the execution path given the tools' output. Each agent is described below

Orchestrator: plans, delegates, and assembles final output using tool calling

Stat Interpreter: analyzes round stats to identify performance patterns, likely scoring leaks, and the highest-priority improvement areas

Mental Game Analyzer: evaluates mindset, emotional control, focus, and decision-making issues using the player debrief and Rotella-style principles

Course Context: pulls course-specific context when available and adds relevant environmental or strategic considerations, with fallback to generic guidance if lookup fails

Practice Plan Generator: turns the combined findings into a structured, actionable practice plan tailored to the golfer's biggest needs

Reviewer: performs quality analysis on the draft plan, checking that it is specific, grounded in the input, and actionable rather than generic

My Evaluation of Good

Good for my project is feedback for a golfer that is novel, actionable, and accurate.

Success is providing the user with what they can improve on and how, with a quality strong enough for them to pursue it. The mental game isn't math, so this isn't objectively

definable, but I tried to be honest when evaluating. The golf course lookup feature was something that could objectively be evaluated, and didn't do overwhelmingly well.

I gave my three test runs an average score of 3.6, accepting the feedback in $\frac{2}{3}$ of the responses. Details can be found in [evaluations/final-submission-evaluation.md](#)

Iteration based on draft feedback

Adding an MCP tool: The course lookup is an MCP tool that fetches real course data and local playing context for the round, giving the agents course-specific information such as location, layout, and conditions when available, with fallback to generic context if no reliable match is found.

What Breaks and Next Steps

The feature that flat-out doesn't work sometimes is the course evaluation. The user inputs the course they played at, and my app calls an api with this string. The nature of this is very fragile. For example, "Tyrone Hills" will return the right course, but "Tyrone Hills Golf Club" may not. Beyond general iteration and refinements, this would be the biggest concrete fix I would make with more time, adding a more robust solution.